



Markdown to PDF Sample Report

Multilingual Rendering Validation

PROJECT

Document Automation PoC

PREPARED FOR

Customer A

Version: 1.0

Date: 2026-08-07

PARTICIPANTS

- Customer Representative A, Platform Engineer, Customer A <customer-a@example.com>
- md2pdf Team, Documentation Engineering, MongoDB <docs@example.com>

Table of Contents

1	Executive Summary	3
2	Key Findings	3
2.1	Document structure	3
2.2	Validation results	3
3	Recommended Delivery Sequence	3

1 Executive Summary

This document is an English sample for validating a document automation workflow. The workflow generates self-contained HTML and PDF files from one Markdown source and checks whether cover metadata, heading hierarchy, tables, code blocks, callouts, and task lists remain readable in PDF.

NOTE

Customer names and values in this document are fictional and exist only for rendering examples.

2 Key Findings

2.1 Document structure

The document presents the main conclusion first, followed by findings and an execution sequence. Long sentences are included to verify that paragraph spacing and line wrapping remain natural when content crosses a page boundary.

Validation item	Current state	Recommended direction
Cover metadata	YAML front matter is used	Keep required fields
Query result table	GFM table is used	Check column widths and wrapping
Code example	JavaScript syntax highlighting is used	Preserve copyable text
Multilingual body	CJK glyphs are tested in companion files	Check for missing glyphs in PDF

The following MongoDB example queries active contracts:

```
db.contracts.find(
  { status: "ACTIVE", region: "US" },
  { _id: 0, contract_id: 1, customer_name: 1 }
).sort({ updated_at: -1 });
```

2.2 Validation results

- HTML title matches the cover title
- Table headers and data rows are clearly separated
- English text and numbers are visible in the PDF
- Performance testing with production-like data is out of scope

3 Recommended Delivery Sequence

1. Review the Markdown front matter and heading hierarchy.
2. Open the HTML first and check links, tables, and code blocks.
3. Inspect every PDF page as it will appear when printed.
4. If a problem is found, update the Markdown or stylesheet instead of the generated files.

This sample demonstrates that four language variants can share the same rendering pipeline.